

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC MÁY TÍNH**



MÔN HỌC: ĐỒ HỌA MÁY TÍNH

**Phép biến đổi affine trong không gian 3D và phép
biến đổi quan sát**

Giảng viên hướng dẫn: TS. CÁP PHẠM ĐÌNH THĂNG

Sinh viên thực hiện: Nguyễn Thế Luân

Mã sinh viên: 23520899

Lớp: CS105.Q22

Thành phố Hồ Chí Minh, tháng 04 năm 2026

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the entire width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the page.

GVHD

TS. Cáp Phạm Đình Thăng

Mục lục

1	Giới thiệu	4
2	Mục tiêu	4
3	Cơ sở lý thuyết	5
3.1	Three.js và đồ họa 3D trên web	5
3.2	Các thành phần cơ bản của scene	5
3.2.1	Scene	5
3.2.2	Camera	5
3.2.3	Renderer	5
3.2.4	Geometry, Material và Mesh	5
3.3	Biến đổi trong không gian 3D (Transform)	5
3.4	Ánh sáng trong Three.js	6
3.5	Camera và hệ tọa độ	6
3.6	Tương tác người dùng	6
4	Nội dung thực hiện	6
4.1	Tổng quan chương trình	6
4.2	Xây dựng giao diện điều khiển	7
4.3	Khởi tạo các biến toàn cục	7
4.4	Khởi tạo scene	8
4.5	Tạo hình hộp	8
4.6	Tạo mặt phẳng	9
4.7	Thêm đối tượng vào scene	9
4.8	Thiết lập đổ bóng	10
4.9	Thiết lập ánh sáng	10
4.9.1	Ambient Light	10
4.9.2	Directional Light	10
4.10	Thêm trục tọa độ	11
4.11	Thiết lập camera	11
4.12	Khởi tạo renderer	11
4.13	Liên kết các điều khiển với sự kiện	12
4.14	Xử lý biến đổi đối tượng	12
4.15	Xử lý camera và LookAt	13
4.16	Cập nhật giá trị hiển thị trên giao diện	14
4.17	Điều chỉnh ánh sáng	14
4.18	Điều chỉnh màu sắc đối tượng và nền	15
4.19	Chức năng reset đối tượng	15
4.20	Chức năng reset camera	16
4.21	Xử lý thay đổi kích thước trình duyệt	16
4.22	Vòng lặp render	16
4.23	Nhận xét	17
4.24	Xây dựng giao diện điều khiển	17
4.25	Khởi tạo Scene và các đối tượng	17
4.26	Thiết lập ánh sáng	18
4.27	Thiết lập Camera	18
4.28	Liên kết giao diện với chương trình	18

4.29	Cập nhật biến đổi đối tượng	18
4.30	Điều khiển Camera và LookAt	19
4.31	Điều chỉnh ánh sáng	19
4.32	Điều chỉnh màu sắc và nền	19
4.33	Render và cập nhật liên tục	19
4.34	Nhận xét	20
5	Phân tích kết quả	20
5.1	Trạng thái ban đầu của scene	20
5.2	Điều chỉnh camera và điểm nhìn	21
5.3	Thực hiện các phép biến đổi đối tượng (Translate, Rotate, Scale)	21
5.3.1	Phép tịnh tiến (Translate):	22
5.3.2	Phép xoay (Rotate):	22
5.3.3	Phép co giãn (Scale):	22
5.4	Điều chỉnh ánh sáng	22
5.5	Thay đổi màu sắc đối tượng và ánh sáng	23
5.6	Thay đổi màu nền và hiệu ứng tổng thể	24
5.7	Nhận xét tổng quát	24
6	Demo và mã nguồn	24

Danh sách hình vẽ

1	Trạng thái ban đầu của scene	20
2	Các nhóm điều khiển LookAt, ánh sáng và màu sắc	21
3	Nhóm điều khiển các phép biến đổi đối tượng	21
4	Kết quả khi thay đổi màu mặt phẳng và màu nền	23
5	Kết quả khi thay đổi màu mặt phẳng và màu nền	24

1 Giới thiệu

Trong các ứng dụng đồ họa 3D hiện đại, yếu tố tương tác đóng vai trò quan trọng giúp người dùng có thể trực tiếp thao tác và quan sát sự thay đổi của đối tượng trong không gian. Trên nền tảng web, thư viện Three.js cung cấp các công cụ mạnh mẽ để xây dựng không chỉ các scene tĩnh mà còn các hệ thống tương tác theo thời gian thực.

Trong bài Lab 05, chương trình được mở rộng từ việc hiển thị scene 3D cơ bản sang xây dựng một môi trường có tính tương tác cao. Người dùng có thể điều chỉnh trực tiếp các thuộc tính của đối tượng như vị trí, góc quay, kích thước, cũng như thay đổi các tham số của camera và ánh sáng.

Bên cạnh đó, bài lab cũng giúp sinh viên hiểu rõ hơn về vai trò của ánh sáng và vật liệu trong việc tạo ra hiệu ứng hiển thị chân thực, cũng như cách các thành phần trong Three.js phối hợp để tạo nên một hệ thống đồ họa hoàn chỉnh.

Thông qua bài thực hành, sinh viên có thể nắm được cách xây dựng một ứng dụng 3D tương tác trên web, làm nền tảng cho các bài toán phức tạp hơn như mô phỏng, game hoặc visualization.

2 Mục tiêu

Bài Lab 05 được thực hiện nhằm đạt được các mục tiêu sau:

- Làm quen với việc xây dựng ứng dụng đồ họa 3D có tính tương tác trên nền web bằng thư viện **Three.js**.
- Hiểu và sử dụng các thành phần cơ bản của một scene 3D:
 - Scene
 - Camera
 - Renderer
 - Mesh (Geometry + Material)
- Xây dựng một scene 3D gồm:
 - Hình hộp (cube)
 - Mặt phẳng (plane)
- Thực hiện các phép biến đổi đối tượng (Transform):
 - Tịnh tiến (Translate)
 - Xoay (Rotate)
 - Co giãn (Scale)
- Điều khiển camera:
 - Thay đổi vị trí camera
 - Điều chỉnh hướng nhìn (lookAt)
- Làm việc với ánh sáng:

- Ambient Light
- Directional Light
- Điều chỉnh cường độ và màu sắc ánh sáng
- Tùy chỉnh vật liệu và màu sắc:
 - Thay đổi màu của đối tượng
 - Thay đổi màu nền của scene
- Xây dựng giao diện điều khiển cho phép người dùng tương tác trực tiếp với scene theo thời gian thực.

3 Cơ sở lý thuyết

3.1 Three.js và đồ họa 3D trên web

Three.js là một thư viện JavaScript mã nguồn mở giúp xây dựng đồ họa 3D trên nền WebGL. Thư viện này cung cấp các công cụ để tạo, quản lý và hiển thị các đối tượng 3D một cách trực quan và dễ dàng hơn so với việc sử dụng WebGL thuần.

3.2 Các thành phần cơ bản của scene

3.2.1 Scene

Scene là không gian chứa toàn bộ các đối tượng 3D, ánh sáng và camera.

3.2.2 Camera

Camera xác định góc nhìn của người quan sát. Trong bài lab sử dụng PerspectiveCamera để mô phỏng góc nhìn thực tế.

3.2.3 Renderer

Renderer có nhiệm vụ hiển thị Scene thông qua Camera lên màn hình. WebGLRenderer tận dụng GPU để tăng hiệu suất xử lý.

3.2.4 Geometry, Material và Mesh

- Geometry: định nghĩa hình dạng đối tượng (Box, Plane,...)
- Material: xác định màu sắc và đặc tính bề mặt
- Mesh: kết hợp Geometry và Material thành đối tượng hoàn chỉnh

3.3 Biến đổi trong không gian 3D (Transform)

Các phép biến đổi cơ bản bao gồm:

- **Translate:** thay đổi vị trí của đối tượng trong không gian
- **Rotate:** xoay đối tượng quanh các trục X, Y, Z

- **Scale:** thay đổi kích thước đối tượng

Các phép biến đổi này được sử dụng để điều khiển object một cách linh hoạt trong scene.

3.4 Ánh sáng trong Three.js

Ánh sáng giúp tăng tính chân thực và chiều sâu cho scene.

- **Ambient Light:** chiếu sáng đều toàn bộ scene
- **Directional Light:** ánh sáng có hướng, tạo hiệu ứng bóng và độ nổi khối

Cường độ và màu sắc ánh sáng ảnh hưởng trực tiếp đến cách object được hiển thị.

3.5 Camera và hệ tọa độ

Hệ tọa độ 3D gồm ba trục:

- X: trái – phải
- Y: lên – xuống
- Z: trước – sau

Camera sử dụng phương thức `lookAt` để xác định hướng nhìn đến một điểm trong không gian.

3.6 Tương tác người dùng

Trong bài lab, các tham số của scene được điều khiển thông qua giao diện người dùng. Người dùng có thể thay đổi các giá trị như vị trí, góc quay, ánh sáng và màu sắc, và kết quả được cập nhật ngay lập tức trên màn hình.

Điều này thể hiện nguyên lý xây dựng các ứng dụng 3D tương tác theo thời gian thực.

4 Nội dung thực hiện

4.1 Tổng quan chương trình

Bài Lab 05 xây dựng một ứng dụng đồ họa 3D tương tác bằng Three.js. Chương trình gồm hai phần chính: giao diện điều khiển được xây dựng bằng HTML/CSS và phần xử lý đồ họa 3D được xây dựng bằng JavaScript. Giao diện cho phép người dùng thay đổi trực tiếp các thuộc tính của đối tượng, camera, ánh sáng và màu sắc trong scene.

Các chức năng chính được triển khai bao gồm:

- Hiển thị một hình hộp trên mặt phẳng.
- Điều khiển phép tịnh tiến, xoay và co giãn của hình hộp.
- Điều khiển vị trí camera và điểm nhìn `LookAt`.
- Điều chỉnh cường độ và màu sắc ánh sáng.
- Thay đổi màu sắc của hình hộp, mặt phẳng và nền.
- Cập nhật kết quả theo thời gian thực.

4.2 Xây dựng giao diện điều khiển

Giao diện điều khiển được đặt trong thẻ `div` có id là `control-panel`. Bảng điều khiển này gồm nhiều nhóm chức năng như biến đổi đối tượng, camera, LookAt, ánh sáng và màu sắc.

Ví dụ, nhóm điều khiển phép tịnh tiến theo trục X được khai báo như sau:

```
<div class="control-item">
  <label>Translate X
    <span class="value" id="txValue">0</span>
  </label>
  <input type="range" id="translateX"
    min="-5" max="5" step="0.1" value="0">
</div>
```

Trong đó:

- `input type="range"` tạo thanh trượt để thay đổi giá trị.
- `id="translateX"` được dùng để JavaScript truy xuất giá trị.
- `span id="txValue"` dùng để hiển thị giá trị hiện tại lên giao diện.

Tương tự, chương trình xây dựng các thanh trượt cho các phép biến đổi còn lại như Translate Y, Translate Z, Rotate X, Rotate Y, Rotate Z và Scale. Ngoài ra, các bộ chọn màu `input type="color"` được dùng để thay đổi màu sắc của đối tượng, mặt phẳng, ánh sáng và nền.

4.3 Khởi tạo các biến toàn cục

Ở đầu chương trình JavaScript, các biến chính được khai báo để lưu trữ scene, camera, renderer, đối tượng và ánh sáng:

```
var scene, camera, renderer;
var box, plane;
var ambientLight, directionalLight;
var lookAtPoint = new THREE.Vector3(0, 0, 0);
```

Các biến này được sử dụng xuyên suốt chương trình. Trong đó:

- `scene`: không gian chứa các đối tượng 3D.
- `camera`: xác định góc nhìn của người quan sát.
- `renderer`: chịu trách nhiệm render scene lên trình duyệt.
- `box`: đối tượng hình hộp.
- `plane`: mặt phẳng nền.
- `ambientLight, directionalLight`: hai nguồn sáng trong scene.
- `lookAtPoint`: điểm mà camera hướng đến.

4.4 Khởi tạo scene

Trong hàm `init()`, scene được tạo bằng đối tượng `THREE.Scene`:

```
scene = new THREE.Scene();
scene.fog = new THREE.FogExp2(0x87ceeb, 0.02);
```

Dòng lệnh đầu tiên tạo không gian 3D chính. Dòng thứ hai thêm hiệu ứng sương mù nhẹ cho scene bằng `FogExp2`. Tham số đầu tiên là màu của fog, tham số thứ hai là mật độ fog. Trong chương trình, mật độ được đặt là 0.02 để tạo hiệu ứng nhẹ, tránh làm mờ quá nhiều đối tượng.

4.5 Tạo hình hộp

Hình hộp được tạo thông qua hàm `getBox()`:

```
function getBox(w, h, d) {
    var geometry = new THREE.BoxGeometry(w, h, d);

    var material = new THREE.MeshStandardMaterial({
        color: 0x4a90e2,
        roughness: 0.35,
        metalness: 0.2
    });

    var mesh = new THREE.Mesh(geometry, material);

    return mesh;
}
```

Trong hàm này:

- `BoxGeometry(w, h, d)` tạo hình hộp với chiều rộng, chiều cao và chiều sâu tương ứng.
- `MeshStandardMaterial` được dùng để vật thể có khả năng phản ứng với ánh sáng.
- `roughness` điều chỉnh độ nhám của bề mặt.
- `metalness` điều chỉnh tính chất kim loại của vật liệu.
- `Mesh` kết hợp geometry và material để tạo thành đối tượng 3D hoàn chỉnh.

Sau khi tạo, hình hộp được đặt lên trên mặt phẳng:

```
box = getBox(1, 1, 1);
box.position.y = 0.5;
```

Do hình hộp có chiều cao là 1, nên đặt `position.y = 0.5` giúp đáy hộp nằm đúng trên mặt phẳng thay vì bị chìm xuống nền.

4.6 Tạo mặt phẳng

Mặt phẳng được tạo bằng hàm `getPlane()`:

```
function getPlane(size) {  
    var geometry = new THREE.PlaneGeometry(size, size);  
  
    var material = new THREE.MeshStandardMaterial({  
        color: 0xe5e7eb,  
        side: THREE.DoubleSide,  
        roughness: 0.95,  
        metalness: 0.0  
    });  
  
    return new THREE.Mesh(geometry, material);  
}
```

Trong đó:

- `PlaneGeometry(size, size)` tạo mặt phẳng hình vuông.
- `side: THREE.DoubleSide` giúp hiển thị cả hai mặt của plane.
- `roughness: 0.95` làm bề mặt ít phản chiếu hơn.
- `metalness: 0.0` giúp mặt phẳng không có tính kim loại.

Sau khi tạo, mặt phẳng được xoay để nằm ngang:

```
plane = getPlane(20);  
plane.rotation.x = Math.PI / 2;
```

Mặc định, `PlaneGeometry` nằm theo phương thẳng đứng. Vì vậy, cần xoay quanh trục X một góc $\pi/2$ radian để mặt phẳng nằm ngang và đóng vai trò làm nền.

4.7 Thêm đối tượng vào scene

Sau khi tạo hình hộp và mặt phẳng, các đối tượng được thêm vào scene:

```
scene.add(box);  
scene.add(plane);
```

Chỉ những đối tượng được thêm vào scene mới có thể được renderer hiển thị trên màn hình.

4.8 Thiết lập đổ bóng

Chương trình bật khả năng đổ bóng cho hình hộp và nhận bóng cho mặt phẳng:

```
box.castShadow = true;
plane.receiveShadow = true;
```

Trong đó:

- `castShadow = true`: cho phép hình hộp tạo bóng.
- `receiveShadow = true`: cho phép mặt phẳng nhận bóng từ đối tượng khác.

Ngoài ra, renderer cũng cần bật chế độ shadow map:

```
renderer.shadowMap.enabled = true;
```

4.9 Thiết lập ánh sáng

Chương trình sử dụng hai nguồn sáng là Ambient Light và Directional Light.

4.9.1 Ambient Light

Ambient Light được dùng để chiếu sáng đều toàn bộ scene:

```
ambientLight = new THREE.AmbientLight(0xffffff, 0.6);
scene.add(ambientLight);
```

Nguồn sáng này không có hướng cụ thể, giúp các vùng tối vẫn có thể nhìn thấy được.

4.9.2 Directional Light

Directional Light mô phỏng ánh sáng có hướng, tương tự như ánh sáng mặt trời:

```
directionalLight = new THREE.DirectionalLight(0xffffff, 1);
directionalLight.position.set(5, 10, 7);
directionalLight.castShadow = true;
scene.add(directionalLight);
```

Trong đoạn code trên:

- `0xffffff`: màu ánh sáng là trắng.
- `1`: cường độ ánh sáng ban đầu.
- `position.set(5, 10, 7)`: đặt nguồn sáng ở phía trên và lệch khỏi đối tượng.
- `castShadow = true`: cho phép nguồn sáng tạo bóng.

4.10 Thêm trục tọa độ

Để hỗ trợ quan sát hệ tọa độ trong không gian 3D, chương trình sử dụng `AxesHelper`:

```
var axesHelper = new THREE.AxesHelper(3);
scene.add(axesHelper);
```

Trục tọa độ giúp người dùng dễ dàng nhận biết hướng của các trục X, Y và Z khi thực hiện các phép biến đổi.

4.11 Thiết lập camera

Camera được tạo bằng `PerspectiveCamera`:

```
camera = new THREE.PerspectiveCamera(
    45,
    window.innerWidth / window.innerHeight,
    1,
    1000
);
```

Các tham số lần lượt là:

- 45: góc nhìn của camera.
- `window.innerWidth / window.innerHeight`: tỉ lệ khung hình.
- 1: khoảng cách gần nhất camera có thể nhìn thấy.
- 1000: khoảng cách xa nhất camera có thể nhìn thấy.

Sau đó, camera được đặt tại vị trí ban đầu:

```
camera.position.set(3, 3, 6);
camera.lookAt(lookAtPoint);
```

Dòng `camera.lookAt(lookAtPoint)` giúp camera luôn hướng về điểm quan sát được chỉ định.

4.12 Khởi tạo renderer

Renderer được tạo bằng `WebGLRenderer`:

```
renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
renderer.setClearColor(0xf3f7fb);
renderer.shadowMap.enabled = true;
```

Trong đó:

- `antialias: true`: làm mượt cạnh của đối tượng.
- `setSize`: thiết lập kích thước vùng hiển thị.

- `setClearColor`: thiết lập màu nền ban đầu.
- `shadowMap.enabled`: bật chế độ đổ bóng.

Sau đó, renderer được gắn vào phần tử HTML có id là `webgl`:

```
document.getElementById("webgl").appendChild(renderer.domElement);
```

4.13 Liên kết các điều khiển với sự kiện

Hàm `bindControls()` có nhiệm vụ liên kết các input trong giao diện với chương trình xử lý:

```
function bindControls() {
    var controls = [
        "translateX", "translateY", "translateZ",
        "rotateX", "rotateY", "rotateZ",
        "scale",
        "cameraX", "cameraY", "cameraZ",
        "lookAtX", "lookAtY", "lookAtZ",

        "ambientLight", "directionalLight",

        "boxColor", "planeColor",
        "ambientColor", "directionalColor",
        "backgroundColor"
    ];

    controls.forEach(function(id) {
        document.getElementById(id)
            .addEventListener("input", applyTransforms);
    });

    document.getElementById("reset-object")
        .addEventListener("click", resetObject);

    document.getElementById("reset-camera")
        .addEventListener("click", resetCamera);

    applyTransforms();
}
```

Trong đoạn code này, mỗi input đều được gắn sự kiện `input`. Khi người dùng kéo thanh trượt hoặc chọn màu, hàm `applyTransforms()` sẽ được gọi để cập nhật scene ngay lập tức.

4.14 Xử lý biến đổi đối tượng

Hàm `applyTransforms()` là phần quan trọng nhất trong chương trình. Đầu tiên, hàm đọc các giá trị liên quan đến phép biến đổi đối tượng:

```

var tx = parseFloat(document.getElementById("translateX").value);
var ty = parseFloat(document.getElementById("translateY").value);
var tz = parseFloat(document.getElementById("translateZ").value);

var rx = parseFloat(document.getElementById("rotateX").value);
var ry = parseFloat(document.getElementById("rotateY").value);
var rz = parseFloat(document.getElementById("rotateZ").value);

var s = parseFloat(document.getElementById("scale").value);

```

Sau đó, các giá trị này được áp dụng lên hình hộp:

```

box.position.set(tx, ty, tz);
box.rotation.set(rx, ry, rz);
box.scale.set(s, s, s);

```

Ý nghĩa:

- `position.set(tx, ty, tz)`: thay đổi vị trí của hình hộp.
- `rotation.set(rx, ry, rz)`: xoay hình hộp quanh các trục X, Y, Z.
- `scale.set(s, s, s)`: thay đổi kích thước hình hộp theo cả ba chiều.

4.15 Xử lý camera và LookAt

Các giá trị camera được đọc từ giao diện:

```

var camX = parseFloat(document.getElementById("cameraX").value);
var camY = parseFloat(document.getElementById("cameraY").value);
var camZ = parseFloat(document.getElementById("cameraZ").value);

```

Sau đó, camera được cập nhật vị trí:

```

camera.position.set(camX, camY, camZ);

```

Tương tự, điểm nhìn LookAt được đọc từ giao diện:

```

var lookX = parseFloat(document.getElementById("lookAtX").value);
var lookY = parseFloat(document.getElementById("lookAtY").value);
var lookZ = parseFloat(document.getElementById("lookAtZ").value);

```

Sau đó, camera được điều chỉnh để hướng về điểm LookAt mới:

```

lookAtPoint.set(lookX, lookY, lookZ);
camera.lookAt(lookAtPoint);

```

Nhờ đó, người dùng có thể thay đổi cả vị trí camera và hướng nhìn của camera trong không gian 3D.

4.16 Cập nhật giá trị hiển thị trên giao diện

Sau khi đọc và áp dụng các giá trị, chương trình cập nhật lại giá trị đang hiển thị trên giao diện:

```
updateValue("txValue", tx);
updateValue("tyValue", ty);
updateValue("tzValue", tz);

updateValue("rxValue", rx);
updateValue("ryValue", ry);
updateValue("rzValue", rz);

updateValue("scaleValue", s);
```

Hàm `updateValue()` được định nghĩa như sau:

```
function updateValue(id, value) {
    document.getElementById(id).textContent = Number(value).toFixed(2);
}
```

Hàm này giúp định dạng giá trị với hai chữ số thập phân, làm giao diện dễ đọc và chuyên nghiệp hơn.

4.17 Điều chỉnh ánh sáng

Cường độ ánh sáng được lấy từ các thanh trượt:

```
var ambientIntensity = parseFloat(
    document.getElementById("ambientLight").value
);

var directionalIntensity = parseFloat(
    document.getElementById("directionalLight").value
);
```

Sau đó, chương trình cập nhật lại cường độ ánh sáng:

```
ambientLight.intensity = ambientIntensity;
directionalLight.intensity = directionalIntensity;
```

Ngoài cường độ, màu sắc ánh sáng cũng được cập nhật:

```
var ambientColor = document.getElementById("ambientColor").value;
var directionalColor = document.getElementById("directionalColor").value;

ambientLight.color.set(ambientColor);
directionalLight.color.set(directionalColor);
```

Điều này cho phép người dùng quan sát sự thay đổi của object khi ánh sáng thay đổi về cả cường độ lẫn màu sắc.

4.18 Điều chỉnh màu sắc đối tượng và nền

Màu của hình hộp, mặt phẳng và nền được lấy từ các input màu:

```
var boxColor = document.getElementById("boxColor").value;
var planeColor = document.getElementById("planeColor").value;
var backgroundColor = document.getElementById("backgroundColor").value;
```

Sau đó, chương trình cập nhật màu cho từng thành phần:

```
box.material.color.set(boxColor);
plane.material.color.set(planeColor);

renderer.setClearColor(backgroundColor);
scene.fog.color.set(backgroundColor);
```

Trong đó:

- `box.material.color.set(boxColor)` thay đổi màu hình hộp.
- `plane.material.color.set(planeColor)` thay đổi màu mặt phẳng.
- `renderer.setClearColor(backgroundColor)` thay đổi màu nền.
- `scene.fog.color.set(backgroundColor)` đồng bộ màu fog với màu nền.

4.19 Chức năng reset đối tượng

Chức năng reset đối tượng đưa hình hộp về trạng thái ban đầu:

```
function resetObject() {
    document.getElementById("translateX").value = 0;
    document.getElementById("translateY").value = 0.5;
    document.getElementById("translateZ").value = 0;

    document.getElementById("rotateX").value = 0;
    document.getElementById("rotateY").value = 0;
    document.getElementById("rotateZ").value = 0;

    document.getElementById("scale").value = 1;

    applyTransforms();
}
```

Sau khi gán lại giá trị mặc định cho các input, hàm `applyTransforms()` được gọi để cập nhật lại scene.

4.20 Chức năng reset camera

Tương tự, chức năng reset camera đưa camera và điểm nhìn về trạng thái mặc định:

```
function resetCamera() {
    document.getElementById("cameraX").value = 3;
    document.getElementById("cameraY").value = 3;
    document.getElementById("cameraZ").value = 6;

    document.getElementById("lookAtX").value = 0;
    document.getElementById("lookAtY").value = 0;
    document.getElementById("lookAtZ").value = 0;

    applyTransforms();
}
```

Chức năng này giúp người dùng nhanh chóng khôi phục lại góc nhìn ban đầu sau khi đã thay đổi camera.

4.21 Xử lý thay đổi kích thước trình duyệt

Khi kích thước cửa sổ trình duyệt thay đổi, camera và renderer cần được cập nhật lại:

```
function onResize() {
    camera.aspect = window.innerWidth / window.innerHeight;
    camera.updateProjectionMatrix();
    renderer.setSize(window.innerWidth, window.innerHeight);
}
```

Hàm này đảm bảo scene không bị méo hình khi người dùng thay đổi kích thước trình duyệt.

Sự kiện resize được đăng ký như sau:

```
window.addEventListener("resize", onResize);
```

4.22 Vòng lặp render

Scene được render liên tục bằng hàm `update()`:

```
function update() {
    renderer.render(scene, camera);
    requestAnimationFrame(update);
}
```

Hàm `requestAnimationFrame()` giúp trình duyệt tự động gọi lại hàm `update()` trước mỗi lần vẽ khung hình mới. Nhờ đó, mọi thay đổi từ giao diện được cập nhật liên tục và mượt mà.

Cuối cùng, chương trình được bắt đầu bằng cách gọi hàm `init()`:

```
init();
```

4.23 Nhận xét

Chương trình đã xây dựng thành công một scene 3D tương tác bằng Three.js. Người dùng có thể điều khiển trực tiếp vị trí, góc quay, kích thước của hình hộp, đồng thời thay đổi vị trí camera, điểm nhìn, ánh sáng và màu sắc.

So với scene 3D tĩnh, chương trình trong Lab 05 thể hiện rõ hơn vai trò của tương tác người dùng trong đồ họa máy tính. Việc kết hợp giữa HTML/CSS và JavaScript giúp tạo ra một ứng dụng trực quan, dễ thao tác và phù hợp để quan sát ảnh hưởng của các phép biến đổi trong không gian 3D.

4.24 Xây dựng giao diện điều khiển

Trong bài Lab 05, một bảng điều khiển (Control Panel) được xây dựng bằng HTML nhằm cho phép người dùng tương tác trực tiếp với scene 3D. Giao diện được thiết kế dưới dạng một panel nằm bên trái màn hình, bao gồm nhiều nhóm chức năng khác nhau.

Cụ thể, các nhóm điều khiển bao gồm:

- Biến đổi đối tượng (Translate, Rotate, Scale)
- Điều khiển camera
- Điều chỉnh điểm nhìn (LookAt)
- Điều chỉnh ánh sáng
- Thay đổi màu sắc đối tượng và nền

Các thành phần điều khiển sử dụng chủ yếu là thanh trượt (input range) để thay đổi giá trị liên tục và bộ chọn màu (input color) để tùy chỉnh màu sắc.

4.25 Khởi tạo Scene và các đối tượng

Scene được khởi tạo để chứa toàn bộ các đối tượng trong không gian 3D:

```
scene = new THREE.Scene();
```

Hai đối tượng chính được sử dụng trong bài lab là hình hộp (cube) và mặt phẳng (plane). Hình hộp được tạo bằng BoxGeometry, trong khi mặt phẳng được tạo bằng PlaneGeometry.

Để các đối tượng có thể tương tác với ánh sáng, chương trình sử dụng MeshStandardMaterial:

```
var material = new THREE.MeshStandardMaterial({  
    color: 0x4a90e2,  
    roughness: 0.35,  
    metalness: 0.2  
});
```

Mặt phẳng được xoay 90 độ để nằm ngang và đóng vai trò làm nền:

```
plane.rotation.x = Math.PI / 2;
```

4.26 Thiết lập ánh sáng

Ánh sáng đóng vai trò quan trọng trong việc hiển thị đối tượng. Trong bài lab, hai loại ánh sáng được sử dụng:

- Ambient Light: chiếu sáng đều toàn bộ scene
- Directional Light: ánh sáng có hướng, tạo hiệu ứng bóng và chiều sâu

```
ambientLight = new THREE.AmbientLight(0xffffff, 0.6);  
directionalLight = new THREE.DirectionalLight(0xffffff, 1);
```

Directional Light được đặt tại vị trí phía trên để chiếu sáng xuống đối tượng:

```
directionalLight.position.set(5, 10, 7);
```

Đồng thời, chế độ đổ bóng được bật để tăng tính chân thực:

```
renderer.shadowMap.enabled = true;
```

4.27 Thiết lập Camera

Camera được sử dụng là PerspectiveCamera với góc nhìn mô phỏng mắt người:

```
camera = new THREE.PerspectiveCamera(  
    45,  
    window.innerWidth / window.innerHeight,  
    1,  
    1000  
);
```

Camera được đặt tại vị trí (3, 3, 6) và hướng về tâm của scene:

```
camera.position.set(3, 3, 6);  
camera.lookAt(new THREE.Vector3(0, 0, 0));
```

4.28 Liên kết giao diện với chương trình

Các thành phần điều khiển trong giao diện được liên kết với chương trình thông qua sự kiện `input`. Khi người dùng thay đổi giá trị, hàm xử lý sẽ được gọi:

```
element.addEventListener("input", applyTransforms);
```

Cách tiếp cận này cho phép cập nhật scene theo thời gian thực.

4.29 Cập nhật biến đổi đối tượng

Hàm `applyTransforms()` đọc các giá trị từ giao diện và áp dụng lên đối tượng:

```
box.position.set(tx, ty, tz);  
box.rotation.set(rx, ry, rz);  
box.scale.set(s, s, s);
```

Nhờ đó, người dùng có thể thay đổi vị trí, góc quay và kích thước của object một cách trực quan.

4.30 Điều khiển Camera và LookAt

Camera cũng được điều chỉnh động thông qua các tham số đầu vào:

```
camera.position.set(camX, camY, camZ);
```

Điểm nhìn của camera được cập nhật thông qua vector LookAt:

```
lookAtPoint.set(lookX, lookY, lookZ);  
camera.lookAt(lookAtPoint);
```

Điều này cho phép thay đổi góc quan sát trong không gian 3D.

4.31 Điều chỉnh ánh sáng

Cường độ ánh sáng được thay đổi thông qua các thanh trượt:

```
ambientLight.intensity = ambientIntensity;  
directionalLight.intensity = directionalIntensity;
```

Ngoài ra, màu sắc của ánh sáng cũng có thể được tùy chỉnh:

```
ambientLight.color.set(ambientColor);  
directionalLight.color.set(directionalColor);
```

4.32 Điều chỉnh màu sắc và nền

Màu sắc của các đối tượng được thay đổi trực tiếp:

```
box.material.color.set(boxColor);  
plane.material.color.set(planeColor);
```

Màu nền của scene được cập nhật thông qua renderer:

```
renderer.setClearColor(backgroundColor);
```

4.33 Render và cập nhật liên tục

Scene được render liên tục bằng cơ chế lặp:

```
function update() {  
    renderer.render(scene, camera);  
    requestAnimationFrame(update);  
}
```

Cơ chế này đảm bảo mọi thay đổi từ người dùng được phản ánh ngay lập tức trên màn hình.

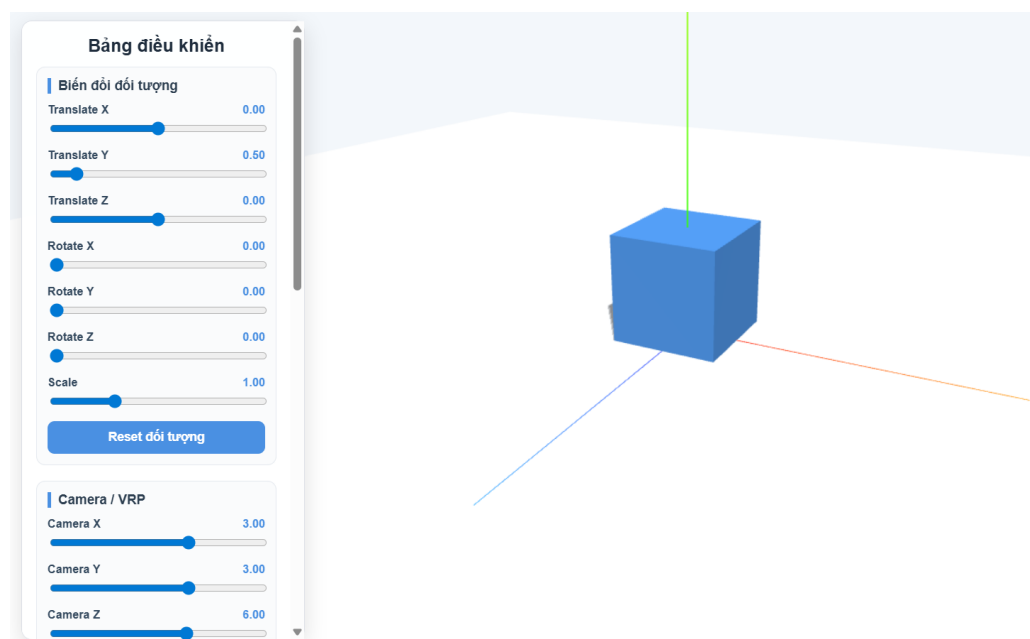
4.34 Nhận xét

Chương trình đã xây dựng thành công một môi trường 3D có tính tương tác cao. Người dùng có thể thay đổi trực tiếp các thuộc tính của đối tượng, camera và ánh sáng, từ đó quan sát được ảnh hưởng của từng yếu tố trong đồ họa 3D.

So với bài Lab trước, bài Lab 05 đã mở rộng đáng kể về mức độ tương tác và khả năng điều khiển scene, giúp sinh viên hiểu rõ hơn về cách xây dựng các ứng dụng đồ họa 3D theo thời gian thực.

5 Phân tích kết quả

5.1 Trạng thái ban đầu của scene



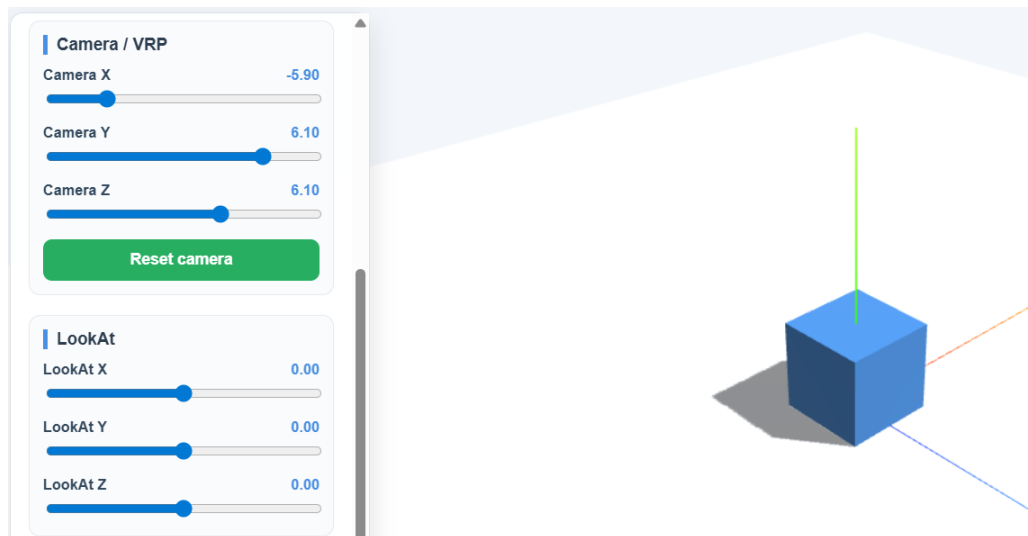
Hình 1: Trạng thái ban đầu của scene

Ở trạng thái ban đầu, hình hộp được đặt tại vị trí trung tâm của mặt phẳng với tọa độ mặc định (0, 0.5, 0). Mặt phẳng đóng vai trò làm nền và được hiển thị rõ ràng dưới đối tượng.

Camera được đặt tại vị trí (3, 3, 6) và hướng về tâm của scene, giúp quan sát toàn bộ đối tượng một cách trực quan. Ánh sáng Ambient Light và Directional Light ở mức mặc định tạo ra độ sáng vừa phải, đảm bảo hình hộp và mặt phẳng đều hiển thị rõ ràng.

Trục tọa độ (AxesHelper) cũng được hiển thị, giúp xác định hướng của các trục X, Y và Z trong không gian 3D.

5.2 Điều chỉnh camera và điểm nhìn

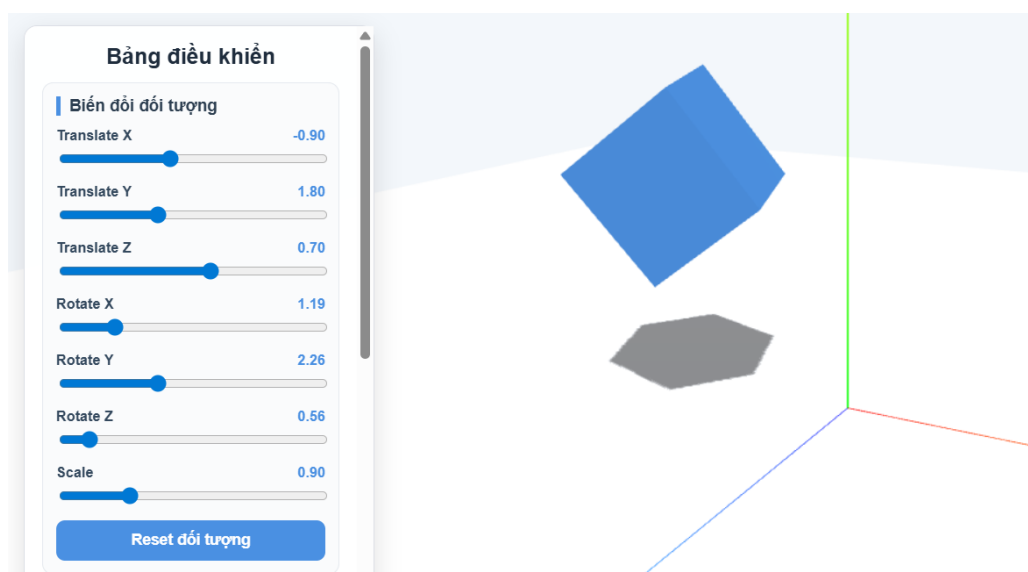


Hình 2: Các nhóm điều khiển LookAt, ánh sáng và màu sắc

Khi thay đổi các tham số của camera và LookAt, góc nhìn của người quan sát thay đổi rõ rệt. Việc điều chỉnh vị trí camera theo các trục X, Y, Z cho phép quan sát đối tượng từ nhiều góc độ khác nhau như từ trên xuống, từ bên cạnh hoặc từ phía trước.

Đồng thời, việc thay đổi điểm LookAt giúp camera tập trung vào các vị trí khác nhau trong không gian, không chỉ giới hạn ở tâm scene. Điều này minh họa rõ ràng cách hoạt động của hệ thống camera trong đồ họa 3D.

5.3 Thực hiện các phép biến đổi đối tượng (Translate, Rotate, Scale)



Hình 3: Nhóm điều khiển các phép biến đổi đối tượng

Trong bài lab, ba phép biến đổi cơ bản trong đồ họa 3D bao gồm tịnh tiến (Translate), xoay (Rotate) và co giãn (Scale) được tích hợp trong cùng một nhóm điều khiển, cho phép người dùng thao tác trực tiếp lên đối tượng.

5.3.1 Phép tịnh tiến (Translate):

Khi thay đổi các giá trị Translate X, Y, Z, hình hộp di chuyển trong không gian theo đúng hướng của từng trục:

- Translate X: di chuyển đối tượng sang trái hoặc phải.
- Translate Y: di chuyển đối tượng lên hoặc xuống.
- Translate Z: di chuyển đối tượng tiến gần hoặc ra xa camera.

Trong thực tế, khi tăng giá trị Translate Y, hình hộp được nâng lên khỏi mặt phẳng, đồng thời bóng đổ xuất hiện rõ bên dưới, cho thấy hệ thống ánh sáng và đổ bóng hoạt động chính xác.

5.3.2 Phép xoay (Rotate):

Các thanh trượt Rotate X, Rotate Y và Rotate Z cho phép xoay đối tượng quanh các trục tương ứng. Việc kết hợp nhiều phép xoay giúp quan sát được toàn bộ các mặt của hình hộp trong không gian 3D.

Phép xoay thể hiện rõ tính chất của hệ tọa độ 3D, trong đó mỗi trục là một trục quay độc lập.

5.3.3 Phép co giãn (Scale):

Thanh trượt Scale cho phép thay đổi kích thước của hình hộp theo tỉ lệ đồng đều trên cả ba trục. Khi giảm giá trị Scale, đối tượng trở nên nhỏ hơn, và khi tăng giá trị, đối tượng lớn hơn tương ứng.

Phép co giãn không làm thay đổi hình dạng mà chỉ ảnh hưởng đến kích thước tổng thể của đối tượng.

Nhận xét:

Ba phép biến đổi trên có thể được sử dụng độc lập hoặc kết hợp với nhau để tạo ra các trạng thái khác nhau của đối tượng. Việc tích hợp các phép biến đổi này trong giao diện điều khiển giúp người dùng dễ dàng quan sát và hiểu rõ hơn về cách đối tượng được thao tác trong không gian 3D.

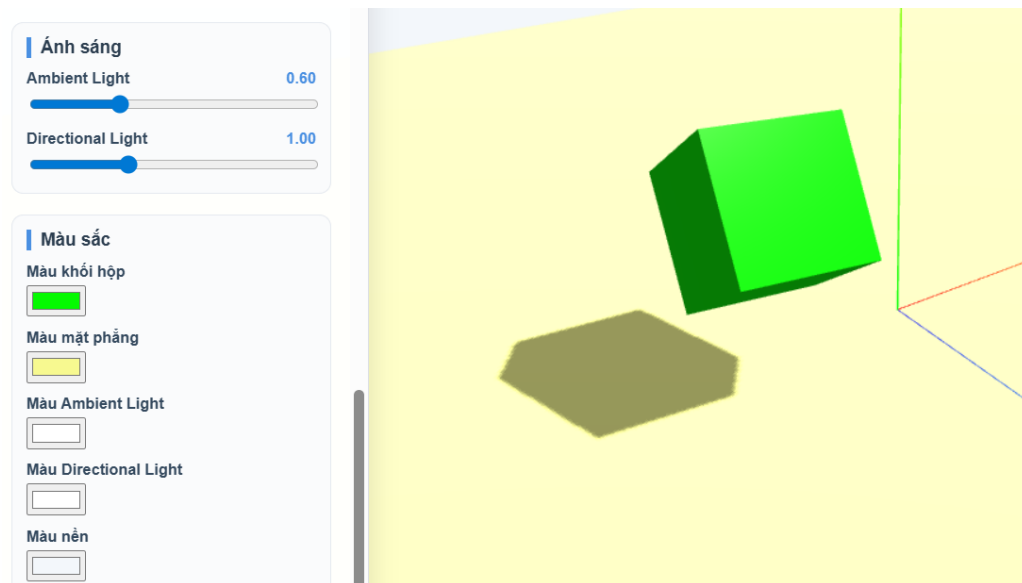
5.4 Điều chỉnh ánh sáng

Khi thay đổi cường độ của Ambient Light và Directional Light, độ sáng của scene thay đổi rõ rệt.

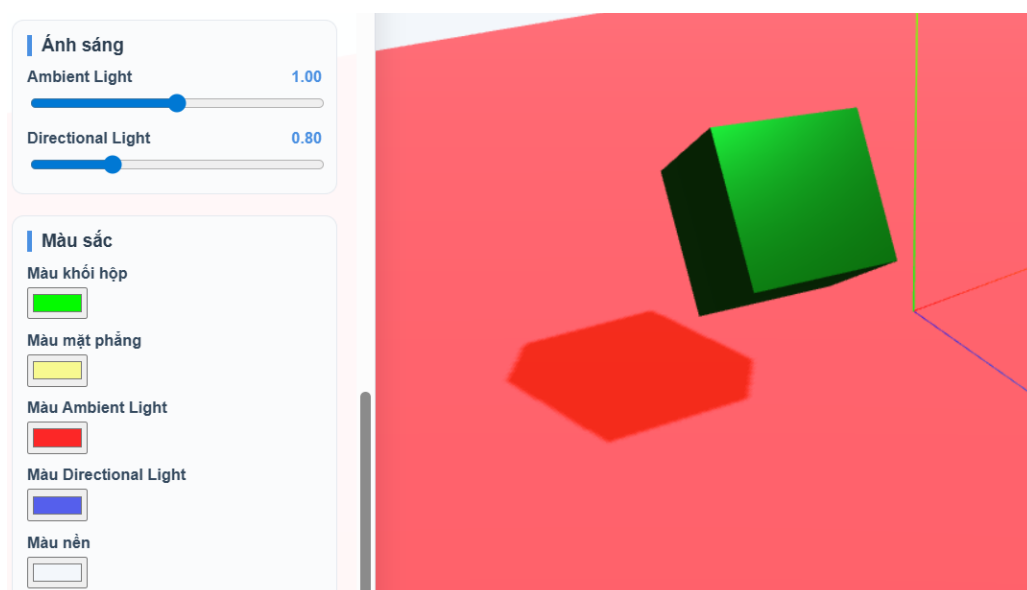
- Ambient Light tăng sẽ làm toàn bộ scene sáng đều hơn.
- Directional Light tăng sẽ làm nổi bật các cạnh và tạo bóng rõ ràng hơn.

Trong hình, khi tăng mạnh ánh sáng, màu sắc của mặt phẳng và bóng đổ trở nên đậm hơn, giúp quan sát rõ sự ảnh hưởng của ánh sáng đến vật thể.

5.5 Thay đổi màu sắc đối tượng và ánh sáng



(a) Thay đổi màu mặt phẳng



(b) Thay đổi màu nền

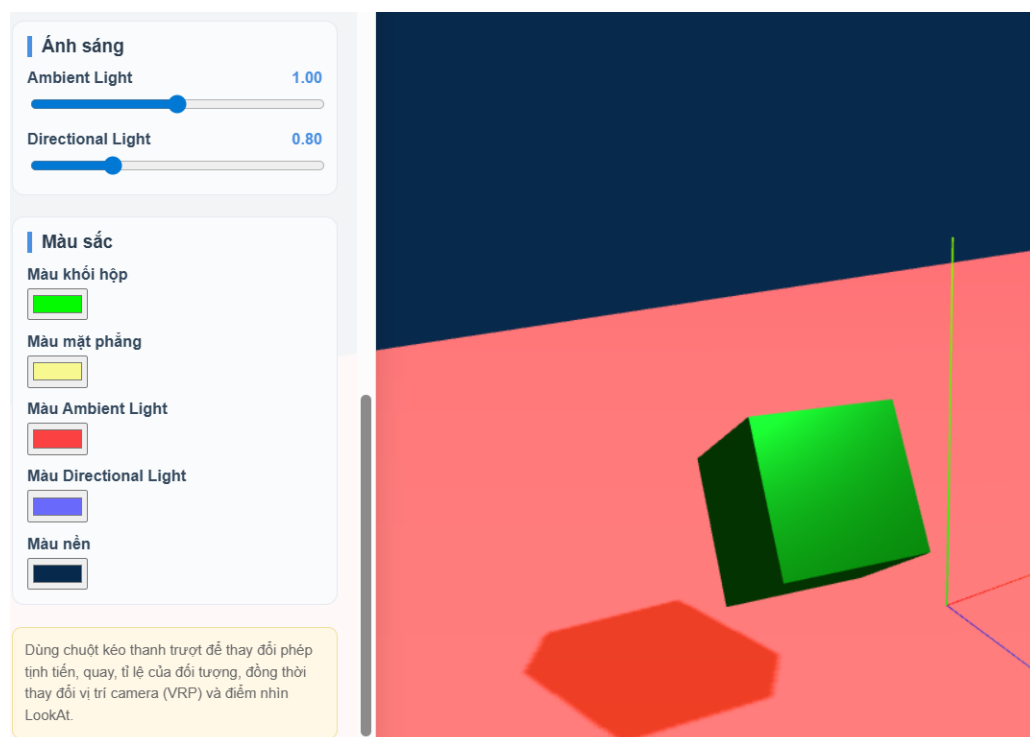
Hình 4: Kết quả khi thay đổi màu mặt phẳng và màu nền

Khi thay đổi màu của hình hộp, mặt phẳng và ánh sáng, scene thay đổi đáng kể về mặt thị giác.

Việc thay đổi màu của ánh sáng (Ambient và Directional) ảnh hưởng trực tiếp đến màu hiển thị của đối tượng. Ví dụ, khi sử dụng ánh sáng màu đỏ hoặc xanh, hình hộp và bóng đổ sẽ bị nhuộm theo màu ánh sáng đó.

Điều này minh họa rõ ràng sự tương tác giữa material và ánh sáng trong mô hình chiếu sáng của Three.js.

5.6 Thay đổi màu nền và hiệu ứng tổng thể



Hình 5: Kết quả khi thay đổi màu mặt phẳng và màu nền

Khi thay đổi màu nền của scene, toàn bộ không gian hiển thị thay đổi theo màu được chọn.

Do chương trình đồng bộ màu nền với màu fog, nên khi thay đổi màu nền, hiệu ứng sương mù cũng thay đổi theo, tạo cảm giác không gian có chiều sâu hơn.

Trong hình cuối, khi nền chuyển sang màu vàng sáng, toàn bộ scene trở nên nổi bật hơn, đồng thời màu sắc của vật thể và bóng đổ cũng bị ảnh hưởng bởi môi trường xung quanh.

5.7 Nhận xét tổng quát

Kết quả cho thấy chương trình hoạt động đúng với các yêu cầu đề ra. Các phép biến đổi đối tượng, điều khiển camera, ánh sáng và màu sắc đều được cập nhật theo thời gian thực và phản ánh chính xác trên scene.

Việc xây dựng giao diện điều khiển giúp người dùng dễ dàng thao tác và quan sát sự thay đổi, từ đó hiểu rõ hơn về các khái niệm trong đồ họa 3D như transform, lighting và camera.

6 Demo và mã nguồn

Để minh họa cho kết quả của bài lab, chương trình đã được triển khai và chia sẻ thông qua các đường dẫn sau:

- **Link web demo:** <https://cs-105-q22-hy1w.vercel.app/>

- **Link GitHub:** CS105.Q22 - Lab-05
- **Video demo:** Lab 05 - Video demo